

MODULE 9 · TROUBLESHOOTING

Node Operations

Take a node out of rotation without losing a pod

Every node eventually needs to come out of rotation

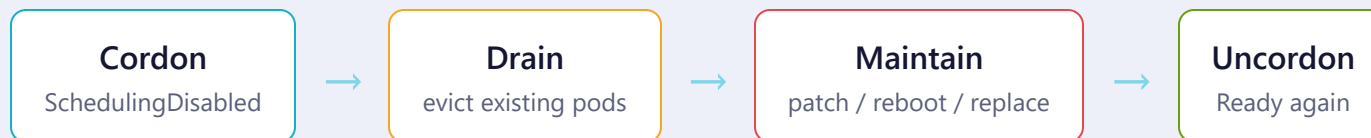
Patching, hardware faults, and decommissioning all mean the same thing: get every pod off a node before you touch it, then bring it back. The lifecycle is **cordon** → **drain** → **maintain** → **uncordon**.

Cordon only stops new work from landing. Drain moves the existing work off — respecting **PodDisruptionBudgets** so a maintenance window doesn't become an outage.

BY THE END YOU CAN

- Cordon a node without disrupting a running pod
- Drain a node the way PodDisruptionBudgets expect
- Explain why --force is needed for bare pods
- Predict DaemonSet behaviour during a drain

Four steps, always in this order



Cordon marks the node; it doesn't move anything. Drain does the evicting, and implies cordon. Uncordon flips scheduling back on but never moves pods back on its own.

Scheduling flag vs. pod placement

COMMAND	EFFECT ON SCHEDULING	EFFECT ON EXISTING PODS
<code>kubectl cordon</code>	Marks the node <code>SchedulingDisabled</code>	None — running pods keep running
<code>kubectl drain</code>	Cordons first, then evicts	Evicts matching pods one at a time, respecting <code>PodDisruptionBudgets</code>
<code>kubectl uncordon</code>	Clears <code>SchedulingDisabled</code>	None — placement is untouched, no auto-rebalance

Eviction, not deletion

- **Eviction API first** — drain calls the eviction subresource per pod, not a raw delete
- **PodDisruptionBudget checked** — the API server rejects an eviction that would breach `minAvailable` / `maxUnavailable`

ORDER MATTERS

A PDB that allows zero unavailable replicas blocks eviction forever — drain just retries every 5s until you fix the PDB or bypass it.

```
$ kubectl drain node01 --ignore-daemonsets --delete-emptydir-data
# Cannot evict pod as it would violate
# the pod's disruption budget (retrying)
```

Two exceptions to "evict everything"

DaemonSet pods are bound to the node by design — evicting one just makes the kubelet recreate it right back. `--ignore-daemonsets` tells drain to leave them running instead of looping forever.

Bare pods — no Deployment / StatefulSet / Job / DaemonSet owner — have no controller to recreate them. Drain refuses to touch them unless you explicitly accept the loss with `--force`.

```
$ kubectl drain node01 --ignore-daemonsets
# DaemonSet pod: left running (owned by node)
# bare pod: error - no controller, use --force

$ kubectl drain node01 --ignore-daemonsets --force
# bare pod: deleted - nothing recreates it
```

--force and --disable-eviction solve different problems

FLAG	WHAT IT DOES	WHEN TO USE
<code>--force</code>	Allows deleting bare pods with no controller owner	Pods from <code>kubect1 run --restart=Never</code> or raw Pod manifests
<code>--disable-eviction</code>	Calls the DELETE API directly, bypassing PodDisruptionBudgets	Single-replica workloads whose PDB would never allow eviction

USE SPARINGLY

Combined, the two flags remove every safety net drain has. Prefer fixing the root cause — scale to 2+ replicas or relax the PDB — over reaching for `--disable-eviction`.

Where people trip

- **Cordon ≠ drain.** Cordon alone leaves every existing pod exactly where it was.
- **A hanging drain isn't stuck.** It's retrying against a PDB it can't satisfy — that's the safety net working as designed.
- **Uncordon doesn't rebalance.** Pods stay put; use rollout restart to spread them back out.
- **--disable-eviction ignores PDBs entirely** — treat it as a last resort, not a default flag to add.

Commands to keep at hand

```
$ kubectl cordon NODE                # stop new scheduling
$ kubectl get nodes                  # look for SchedulingDisabled
$ kubectl drain NODE --ignore-daemonsets --delete-emptydir-data # evict, respecting PDBs
$ kubectl drain NODE ... --force      # also remove bare/unmanaged pods
$ kubectl drain NODE ... --disable-eviction # bypass PDBs (last resort)
$ kubectl uncordon NODE                # resume scheduling
```

Tip: drain = cordon + evict. Nothing moves pods back — that's on you (rollout restart).

RECAP

Node maintenance in one breath

- Cordon marks a node unschedulable; running pods are untouched
- Drain evicts existing pods through the eviction API, respecting PodDisruptionBudgets
- Bare pods need `--force`; DaemonSet pods are skipped with `--ignore-daemonsets`
- Uncordon restores scheduling but never rebalances pods back

HANDS-ON

Now run Lab 9.10
[labs/9.10/](#)

Next: [9.11 — Logs, Events & Metrics](#)